

Hack-AI-thon Screening Assignment

Overview

Please complete all sections below and submit your responses in a single document along with the required links.

1. LLM Project

Describe one project where you used an LLM API in a non-trivial way. This should not be a tutorial clone or simple chatbot wrapper.

- What problem the system solved
- The high-level architecture and key components
- One engineering challenge that was difficult or unexpected

If you have not built an LLM application before, do not fabricate experience. Instead, describe the most technically complex system you have done and be honest about what you haven't built yet.

Please include:

- Public GitHub repository link with clear setup instructions in the README file
- Short demo video

2. Architecture Design

Your team needs to build a document Q&A system over a corpus of 10,000 internal documents (mix of PDFs, Word files, and Confluence pages).

Users ask natural language questions and expect accurate, cited answers. Basic keyword search already exists but fails on semantic queries.

Design the pipeline.

- What you'd use for chunking and why
- Embedding model choice
- Vector store choice
- Retrieval strategy

- How you'd handle documents that answer the question only partially

Also explain:

- What is the biggest failure mode in your design?
- How would you mitigate it?

3. Debug Challenge — Find and Fix the Bugs

The code below is a production RAG pipeline a junior engineer submitted for review. It has multiple bugs — some will cause runtime crashes, some will silently produce wrong answers, and some are architectural mistakes that will only surface under load.

Identify as many issues as you can, explain what each bug actually causes (not just “this is wrong”), and provide the corrected version for each one.

```
from langchain.document_loaders import TextLoader
from langchain.text_splitter import CharacterTextSplitter
from langchain.embeddings import OpenAIEmbeddings
from langchain.vectorstores import Chroma
from langchain.chains import RetrievalQA
from langchain.chat_models import ChatOpenAI

# Load documents
loader = TextLoader("company_docs.txt")
docs = loader.load()

# Split into chunks
splitter = CharacterTextSplitter(
    chunk_size=100,
    chunk_overlap=500
)
chunks = splitter.split_documents(docs)

# Create embeddings
embeddings = OpenAIEmbeddings(model="gpt-4")
vectorstore = Chroma.from_documents(chunks, embeddings)

# Build retrieval chain
llm = ChatOpenAI(model="gpt-4", temperature=1.0)

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=vectorstore.as_retriever(
        search_kwargs={"k": 50}
    ),
```

```
        return_source_documents=False
    )

    # Query - no error handling
    result = qa_chain.run("What is our refund policy?")
    print(result)
```